



**Universidad Nacional Autónoma de
México
Facultad de Ingeniería**



Proyecto Final: “Caso Práctico 5”

**Alumnos: Hernández Ruiz Leny Javier
Silverio Martínez Andrés
Equipo: Los Soles de México**

Materia: Administración de Servicios de Internet

Grupo: 3

Semestre: 2026-2

Equipo: 7

Profesor: Ángel Brito Segura

Fecha de entrega: 26-05-26

Proyecto Final

Introducción

En el entorno actual de las tecnologías de la información, la infraestructura de servicios de Internet no solo debe garantizar la conectividad, sino también la resiliencia operativa, la seguridad de la información y la escalabilidad. El presente proyecto documenta la implementación integral de una arquitectura de red de grado empresarial, diseñada para resolver los requerimientos técnicos y operativos del "Caso Práctico 5" de la asignatura Administración de Servicios de Internet.

El objetivo central de este desarrollo fue construir un ecosistema de servicios robusto, compuesto por una infraestructura de alta disponibilidad (basada en un clúster Master-Slave con Keepalived y HAProxy) y una gestión unificada de identidades mediante OpenLDAP. Asimismo, se integraron soluciones de almacenamiento colaborativo en la nube con Nextcloud y PostgreSQL, complementadas con un acceso remoto seguro mediante un túnel VPN de última generación con WireGuard.

Más allá de la configuración de servicios aislados, el valor de este proyecto reside en la interconexión transparente de sus componentes: desde la automatización mediante contenedores de Docker hasta la sincronización en tiempo real de permisos corporativos. La infraestructura resultante no solo cumple con los estándares de redundancia ante fallos, sino que ofrece una solución soberana y eficiente frente a las alternativas comerciales, demostrando cómo una arquitectura bien diseñada puede garantizar la integridad y disponibilidad de los activos digitales en una firma de arquitectura moderna.

Objetivos

- Diseñar e implementar una arquitectura de red resiliente y escalable para la administración de servicios de Internet, que garantice la alta disponibilidad de recursos corporativos mediante un clúster de servidores, la centralización de identidades y el acceso seguro desde redes externas.
- Implementar un esquema de redundancia Master-Slave mediante el servicio Keepalived y un balanceador de carga, asegurando que el acceso a los servicios internos se mantenga constante bajo una IP Virtual (VIP) ante cualquier fallo en el nodo principal.
- Configurar y desplegar un servicio de directorio activo basado en OpenLDAP, permitiendo una administración unificada y coherente de las cuentas de usuario y niveles de acceso a través de toda la infraestructura.
- Lograr la interoperabilidad total entre el directorio LDAP y la plataforma de nube privada Nextcloud, mediante la correcta asignación dinámica de grupos y permisos corporativos, eliminando la duplicidad de cuentas y la gestión manual de archivos.
- Establecer un túnel VPN seguro utilizando el protocolo WireGuard, proporcionando a los residentes en obra un acceso transparente y encriptado a la Intranet, cumpliendo con los estándares actuales de seguridad en comunicaciones de red.
- Definir un modelo de despliegue escalable en la nube que separe el procesamiento de datos del almacenamiento y la gestión de bases de datos, logrando una arquitectura más eficiente y económicamente sostenible en comparación con soluciones de licenciamiento comercial.
-

Análisis de requerimientos

Funcionales

Para la implementación de esta infraestructura, definimos los siguientes requerimientos que el sistema debe cubrir para operar de manera exitosa:

Funcionales	Descripción
Gestión Centralizada de Identidades	El sistema debe autenticar a todos los usuarios de la nube privada (Nextcloud) consultando directamente una base de datos de directorio activo gestionado por OpenLDAP.
Asignación Dinámica de Permisos	El sistema debe asignar automáticamente a los usuarios a sus respectivas carpetas de equipo basándose en su pertenencia a grupos en LDAP (utilizando el atributo member para garantizar la compatibilidad con el entorno Linux).
Alta Disponibilidad (HA)	La arquitectura debe mantener el portal web accesible en todo momento bajo una única IP Virtual (VIP 192.168.12.250). En caso de que el servidor Master falle, el servidor Slave debe heredar la IP y mantener el servicio en línea de forma transparente.
Acceso Remoto Seguro	El sistema debe permitir el acceso a la intranet desde redes externas mediante la implementación de un túnel VPN con WireGuard, configurado a través del puerto UDP 51820 en el router de frontera.
Aprovisionamiento Gráfico de Usuarios	La arquitectura debe soportar la creación, modificación y asignación de usuarios a través de una interfaz de gestión gráfica (GUI), eliminando la necesidad de interactuar directamente con la línea de comandos para agilizar la operación diaria de recursos humanos.

No funcionales

No Funcionales	Descripción
Tolerancia a fallos	El tiempo de conmutación (failover) del servicio Keepalived entre los nodos debe ejecutarse en menos de 2 segundos, garantizando una interrupción mínima en el servicio.
Seguridad y Aislamiento	Los servicios internos no deben estar expuestos directamente a Internet. Toda conexión externa debe pasar obligatoriamente por la encriptación de la VPN.
Portabilidad	Los servicios principales (Nextcloud y OpenLDAP) deben ejecutarse como contenedores de Docker, asegurando que la infraestructura sea replicable rápidamente
Rendimiento	El servidor web (Nginx) debe actuar como proxy inverso para despachar las peticiones hacia los contenedores de forma eficiente.

Arquitecturas

Diagrama de arquitectura para Producción con análisis de costos de implementación y mantenimiento.

Para el despliegue en producción, calculamos un presupuesto inicial basado en proveedores en la nube enfocado en una constructora PYME de 50 empleados.

Tabla 1: Desglose de Costos de Infraestructura y Mantenimiento

Categoría	Concepto/Descripción Técnica	Costo Anual (USD)
Infraestructura	VPS Master (Cómputo): Servidor con 4GB RAM y 2 Vcpu.	\$288
	VPS Slave (Cómputo/LDAP): Servidor con 2GB RAM y 1 Vcpu.	\$144
	Almacenamiento Externo (S3 o Block Storage): 250 GB para las carpetas compartidas corporativas.	\$60
	Base de Datos Gestionada (PostgreSQL): Para mejorar el rendimiento y búsquedas de la nube.	\$180
	Dominio y Certificados SSL: Dominio corporativo (Certificados Let's Encrypt gratuitos).	\$15

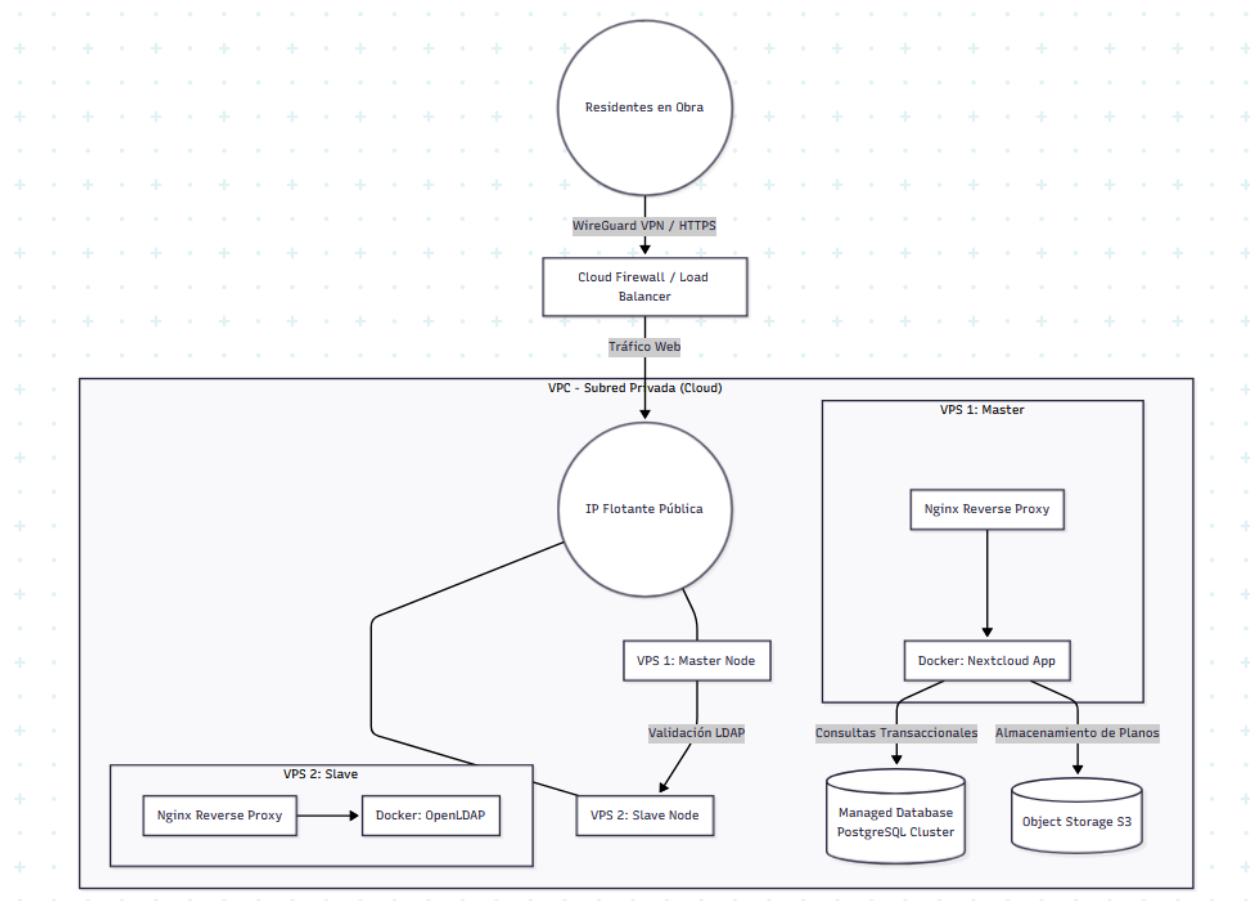
	Subtotal de Infraestructura	\$687
Mantenimiento Operativo	Horas de Administración SysAdmin: 5 horas mensuales para monitoreo, respaldos y actualización de contenedores (Tarifa base: \$25 USD / hora = \$125 USD/mes).	\$1500
	TOTAL COSTO OPERATIVO ANUAL	\$2187

Tabla 2: Retorno de Inversión (ROI) y Comparativa

Escenario de Implementación	Descripción del Esquema	Costo Anual (USD)
Plataformas Comerciales (Terceros)	Pago de licencias por usuario (\$72 USD anuales aprox.) estimado para una plantilla de 50 empleados.	\$3600
Arquitectura Propuesta (Software Libre)	Infraestructura altamente disponible, gráficamente administrable y con soberanía total sobre los datos.	\$687
Ahorro Directo (Anual)	Reducción de costos por evitar licenciamiento corporativo.	\$2913

Para migrar este proyecto a un entorno de producción real, la arquitectura prescinde del módem doméstico, trasladándose a proveedores de nube pública (IaaS) con bases de datos externas para mayor robustez.

El diagrama es el siguiente:

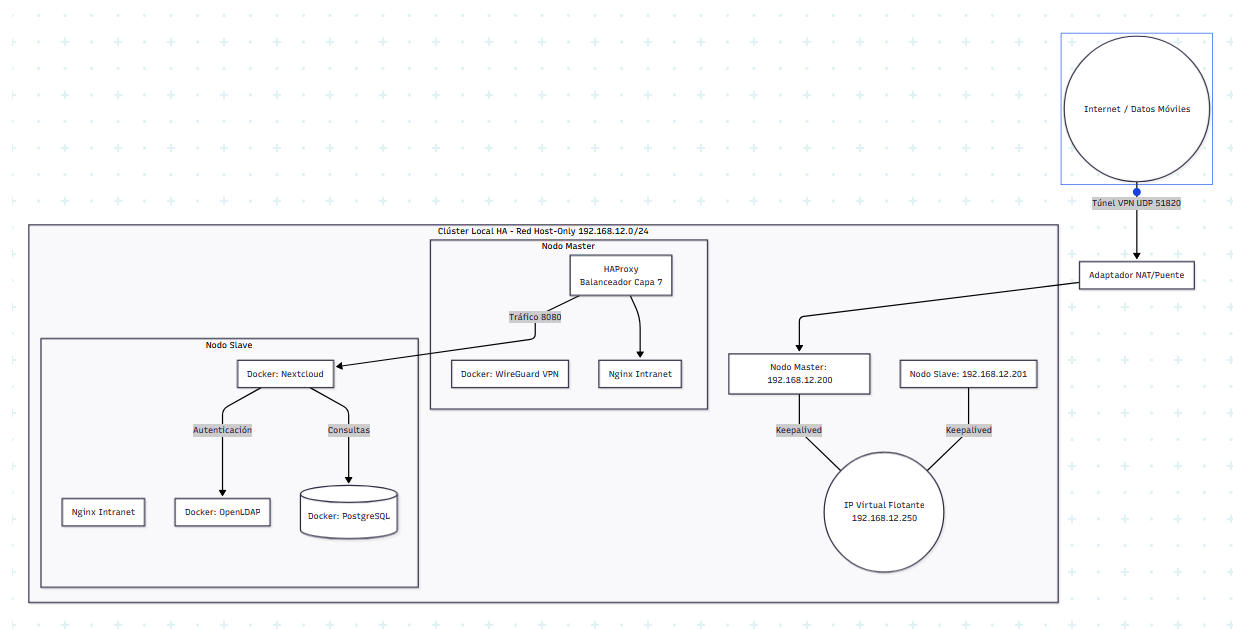


Por su parte, el Diagrama de Arquitectura para Producción representa el despliegue a escala real dentro de una infraestructura en la nube (VPC), estructurado para soportar una alta concurrencia de usuarios sin cuellos de botella. En este modelo, el tráfico encriptado de los trabajadores de la constructora pasa por un firewall y un balanceador de carga en la nube que dirige las peticiones hacia el clúster de servidores privados. La principal evolución técnica de este entorno es la descentralización de los recursos pesados: el procesamiento de Nextcloud se separa del almacenamiento físico enviando los planos a un servicio de almacenamiento de objetos (S3), mientras que las consultas se delegan a un clúster de bases de datos PostgreSQL autónomo y gestionado, garantizando que el sistema sea infinitamente escalable y la información corporativa jamás se pierda.

Diagrama de arquitectura del ambiente de Desarrollo

En esta fase, construimos el clúster utilizando virtualización local para simular las condiciones de una red empresarial detrás de un ISP doméstico.

El diagrama es el siguiente:



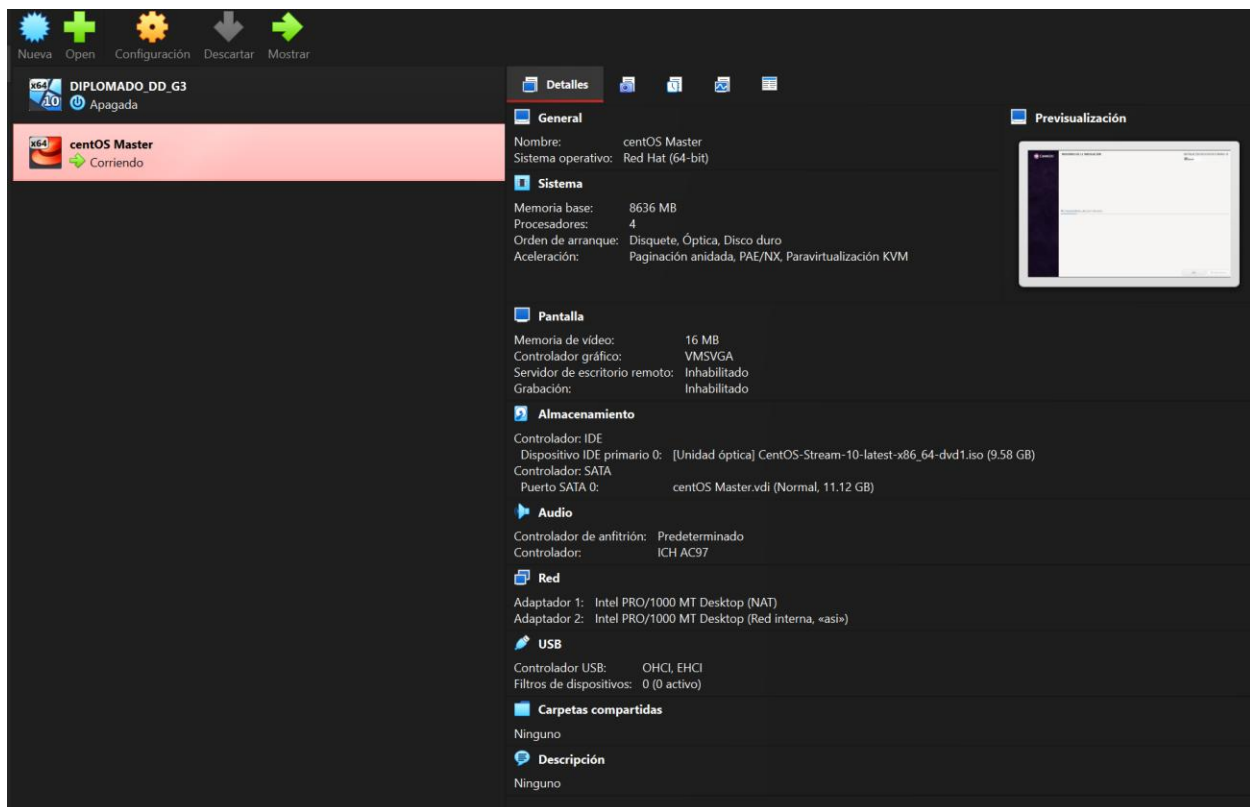
El Diagrama de Arquitectura del Ambiente de Desarrollo ilustra una topología de red simulada localmente mediante máquinas virtuales, diseñada para replicar las condiciones de una infraestructura empresarial tolerante a fallos. En este entorno, el tráfico exterior ingresa a través de un módem doméstico con reenvío de puertos hacia un clúster blindado en una red privada aislada (192.168.12.X). La alta disponibilidad se garantiza mediante el servicio Keepalived, que administra una IP virtual flotante (VIP) entre el nodo Master (encargado de la VPN con WireGuard y el proxy inverso) y el nodo Slave (que gestiona la base de datos maestra de OpenLDAP y la interfaz gráfica). Ambos nodos ejecutan contenedores de Nextcloud y Nginx, permitiendo que la intranet y la nube sigan operando de forma ininterrumpida incluso si el servidor principal sufre una caída.

Bitácora de configuraciones

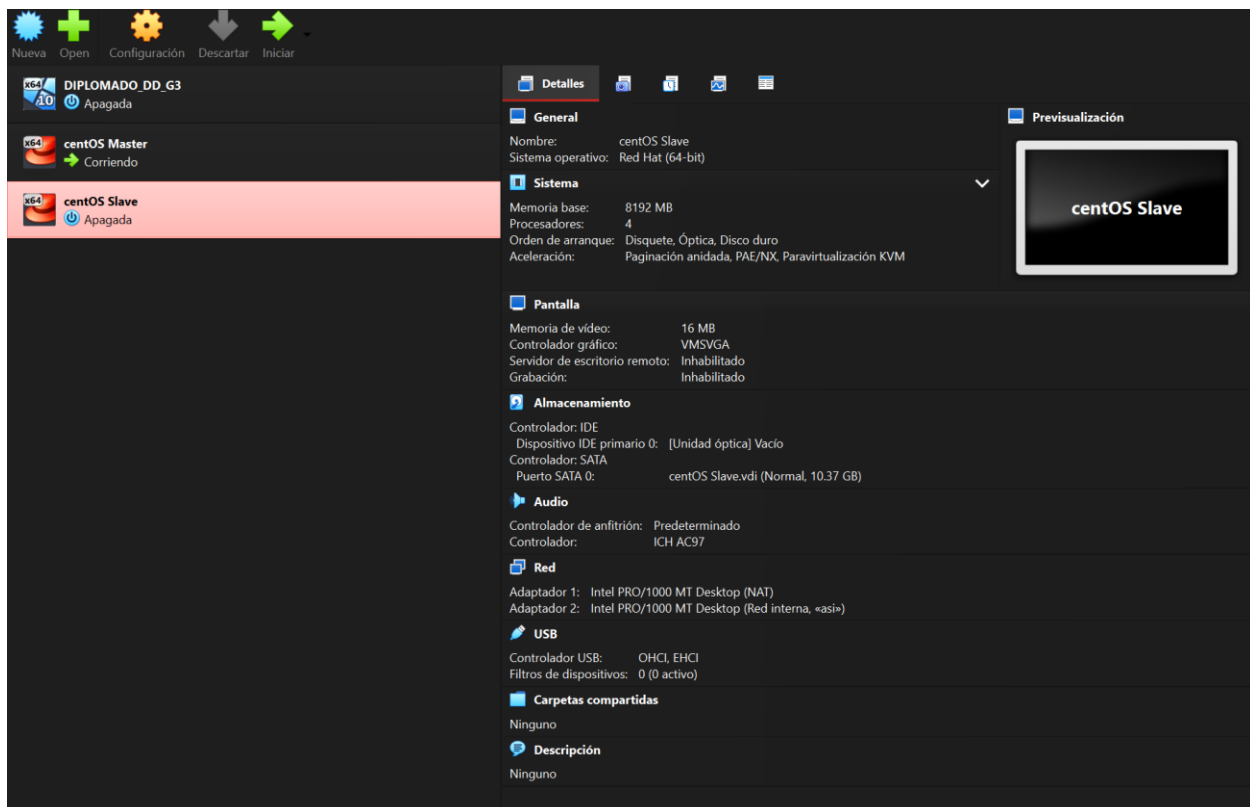
Para la implementación del proyecto, se optó por el uso de máquinas virtuales, ya que son de fácil uso, y no requiere de muchas configuraciones complicadas para arrancarlas y hacer que el proyecto funcione de manera correcta.

Para ello, se crearon dos máquinas virtuales, la “master” (la cual simulará nuestro servidor), y la “slave” (la cual simulará ser nuestro cliente). Para ello, ocupamos tener la imagen del sistema correspondiente a “centOS Stream 10”, así como el entorno donde se realizará la instalación (en este caso, ORACLE VirtualBox).

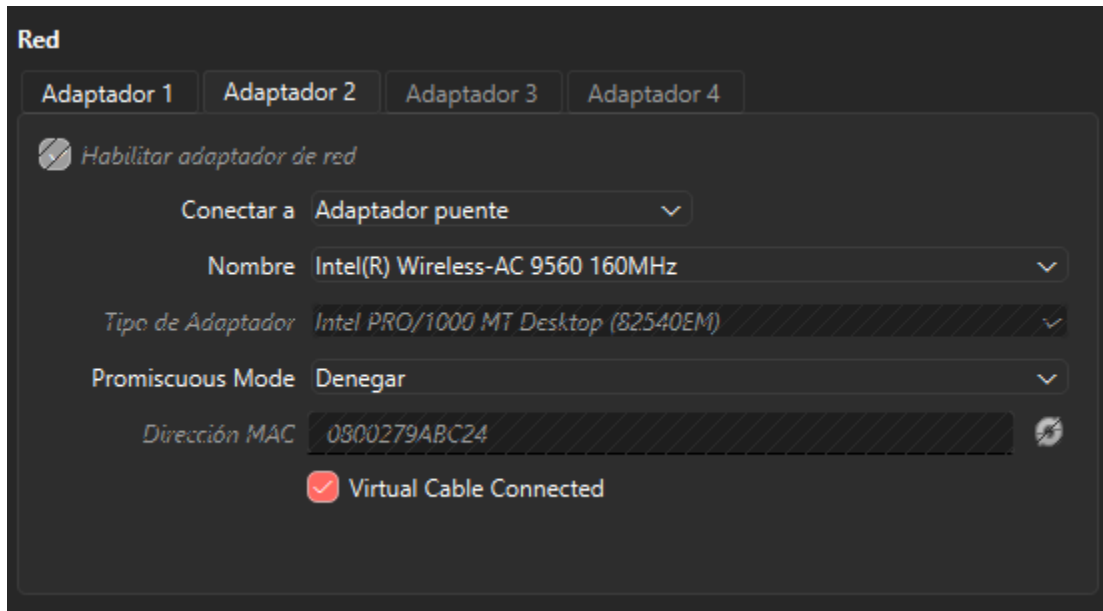
Donde, para la primer máquina virtual (master), se configuró con las siguientes propiedades:



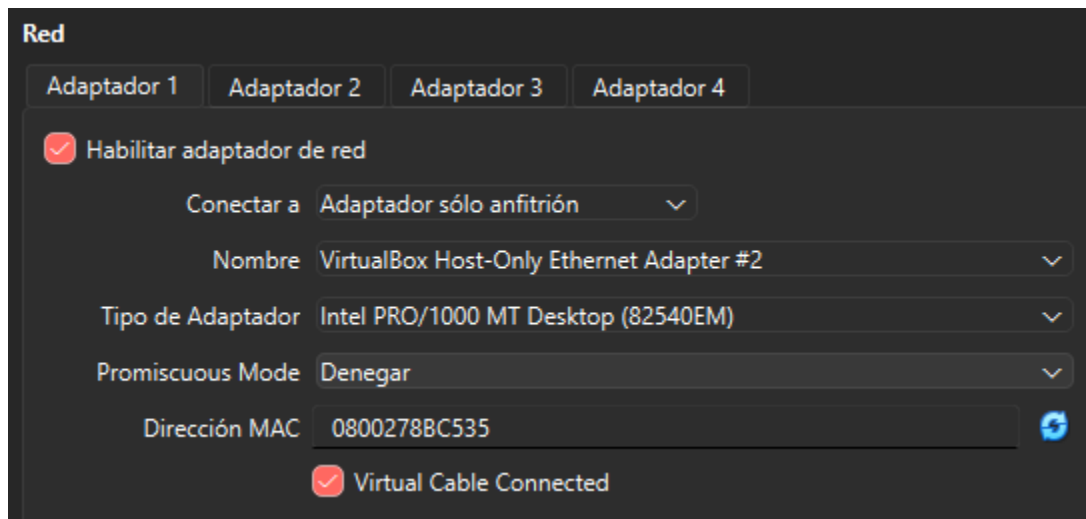
De manera similar, la configuración para la máquina virtual “slave”, es la siguiente:



En cuanto a la configuración de la red, se habilitó el segundo adaptador de red, esto para lograr la conexión entre máquinas virtuales, se seleccionó la opción de “adaptador puente”, con el fin de poder conectarnos desde cualquier dispositivo a la página web.



Mientras que al primer adaptador de red, para que las direcciones IP no causaran mayor problema, se les configuró un rango de direcciones IP, hiendo de 192.168.12.1 - 192.168.12.254.



Si bien, aunque ya se hayan configurado ambos adaptadores, en cada una de las máquinas virtuales había que asignarles una IP fija para que ambas máquinas virtuales pudieran establecer conexión, poniéndoles las siguientes direcciones IP:

- Master

Detalles
Identidad
IPv4
IPv6
Seguridad

Método IPv4

☐ Automático (DHCP)
☐ Sólo enlace local

☒ Manual
☐ Desactivar

☐ Compartida con otros equipos

Direcciones

Dirección	Máscara de red	Puerta de enlace	
192.168.12.200	255.255.255.0	192.168.12.72	✕
			✕

- Slave

Detalles
Identidad
IPv4
IPv6
Seguridad

Método IPv4

☐ Automático (DHCP)
☐ Sólo enlace local

☒ Manual
☐ Desactivar

☐ Compartida con otros equipos

Direcciones

Dirección	Máscara de red	Puerta de enlace	
192.168.12.201	255.255.255.0	192.168.12.72	✕
			✕

Donde, para confirmar que ambas máquinas virtuales ya se estaban reconociendo una a otra, se hizo ping entre una y otra.

```

master@master:~/PFASI$ ping 192.168.12.201
PING 192.168.12.201 (192.168.12.201) 56(84) bytes of data.
64 bytes from 192.168.12.201: icmp_seq=1 ttl=64 time=1.47 ms
64 bytes from 192.168.12.201: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.12.201: icmp_seq=3 ttl=64 time=2.46 ms
64 bytes from 192.168.12.201: icmp_seq=4 ttl=64 time=2.96 ms
^C
--- 192.168.12.201 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 1.469/2.409/2.964/0.571 ms

```

Para empezar, en la máquina virtual “master”, se implementarán los servicios Keepalived (el cual nos ayuda a manejar una IP virtual por si es que se llega a caer el servidor), y HAProxy (el cual

ayudará a balancear la carga de las peticiones web). Para ello, creamos el siguiente script secuencial que implementará ambos servicios.

```
#!/bin/bash

# --- VARIABLES ---
INTERFAZ="enp0s8"
VIP="192.168.12.250"
IP_NODO2="192.168.12.201"

echo "Actualizando el sistema e instalando paquetes base..."
dnf update -y

dnf install -y keepalived haproxy nano

echo "Configurando Keepalived..."
cat <<EOF > /etc/keepalived/keepalived.conf
vrrp_instance VI_1 {
    state MASTER
    interface $INTERFAZ
    virtual_router_id 51
    priority 101
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    virtual_ipaddress {
        $VIP
    }
}
EOF

echo "Configurando HAProxy..."
# Se realiza un respaldo del archivo original por cualquier cosa
cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bak

# Se configura HAProxy para que escuche en la VIP y mande el
tráfico al Nodo 2
cat <<EOF > /etc/haproxy/haproxy.cfg
global
    log          127.0.0.1 local2
    chroot       /var/lib/haproxy
    pidfile      /var/run/haproxy.pid
    maxconn      4000
    user         haproxy
    group        haproxy
    daemon

defaults
    mode          http
    log           global
    option        httplog
    option        dontlognull
    retries       3
```

```

        timeout http-request      10s
        timeout queue             1m
        timeout connect           10s
        timeout client            1m
        timeout server            1m

frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server nodo2_web $IP_NODO2:80 check
EOF

echo "Abriendo puertos en el Firewall (HTTP y VRRP para
Keepalived)..."
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-protocol=vrrp
firewall-cmd --reload

echo "Habilitando e iniciando los servicios de manera limpia..."
systemctl enable keepalived
systemctl start keepalived
systemctl enable haproxy
systemctl start haproxy

```

Pero, para que el servicio siga vivo si es que se cae el master, keepalived también se instaló en la máquina slave, solo que sin la configuración de haproxy, ya que no es necesario para que este funcione.

```

#!/bin/bash

# 1. Actualizar repositorios e instalar el servicio
echo "[1/4] Actualizando paquetes e instalando Keepalived..."
sudo apt update -y && sudo apt install keepalived -y

# 2. Inyectar la configuración limpia usando 'tee' para respetar
permisos de sudo
cat <<EOF | sudo tee /etc/keepalived/keepalived.conf > /dev/null
vrrp_instance VI_1 {

```

```

state BACKUP
interface enp0s8
virtual_router_id 51
priority 90
advert_int 1
authentication {
    auth_type PASS
    auth_pass 1111
}
virtual_ipaddress {
    192.168.12.250
}
}
EOF

# 3. Habilitar el servicio para que inicie con el sistema y
arrancarlo
sudo systemctl enable keepalived
sudo systemctl restart keepalived

# 4. Imprimir el estado final para confirmar que todo salió bien
sudo systemctl status keepalived --no-pager | head -n 10

```

Ahora bien, después de instalados dichos servicios, y de haber hecho la corroboración de que están funcionando de manera correcta, se instala el siguiente servicio, el cual es nginx en la máquina slave, ya que se solicita una intranet para poder visualizar exclusivamente las direcciones de la empresa. El instalar nginx con un módulo de autenticación básica (en este caso htpasswd) ayudará a realizar este requerimiento.

Entonces, el script secuencial para la instalación de nginx, nos queda de la siguiente manera:

```

#!/bin/bash

echo "Instalando Nginx y utilidades de contraseñas..."
dnf update -y
dnf install -y nginx httpd-tools

echo "Creando el directorio de la Intranet y la página de
cronogramas..."
mkdir -p /usr/share/nginx/html/intranet
cat <<EOF > /usr/share/nginx/html/intranet/index.html

```

Seguido de esto, se implementó OpenLDAP en la máquina slave, esto con el propósito de gestionar un directorio central con los contactos y permisos de todos los contratistas involucrados. El script secuencial queda de la siguiente manera:

```
#!/bin/bash

echo "Instalando Docker y clientes LDAP..."
dnf config-manager
--add-repo=https://download.docker.com/linux/centos/docker-ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io
openldap-clients
dnf install -y docker-ce docker-ce-cli containerd.io
openldap-clients --nogpgcheck

echo "Iniciando el demonio de Docker..."
systemctl enable --now docker

echo "Desplegando el contenedor de OpenLDAP..."
docker rm -f servidor ldap 2>/dev/null

docker run -d \
  -p 389:389 -p 636:636 \
  --name servidor ldap \
  --restart always \
  -e LDAP_ORGANISATION="Firma Arquitectura" \
  -e LDAP_DOMAIN="arquitectura.local" \
  -e LDAP_ADMIN_PASSWORD="admin123" \
  osixia/openldap:1.5.0

echo "Esperando 15 segundos a que el contenedor inicialice la base de datos..."
sleep 15

echo "Creando archivo de Unidades Organizativas (OUs)..."
cat <<EOF > ous.ldif
dn: ou=contratistas,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: contratistas

dn: ou=directores,dc=arquitectura,dc=local
objectClass: organizationalUnit

ou: directores

dn: ou=residentes,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: residentes
EOF

echo "Inyectando la jerarquía en el contenedor..."
ldapadd -x -H ldap://localhost:389 -D
"cn=admin,dc=arquitectura,dc=local" -w admin123 -f ous.ldif

echo "Abriendo puerto LDAP en el firewall..."
firewall-cmd --permanent --add-service=ldap
firewall-cmd --reload
```

Se tuvo que implementar mediante Docker, ya que había un pequeño problema con las dependencias instaladas en la versión de CentOS 10, sin embargo, mediante Docker funciona de la misma manera y, a nivel empresarial, su implementación es mejor.

Al hacer la comprobación de la creación de las carpetas jerárquicas que se crearon, se pueden ver de la siguiente manera (confirmando su correcta implementación):

```

slave@slave:~/PFASI$ ldapsearch -x -H ldap://localhost:389 -b "dc=arquitectura,d
c=local" -D "cn=admin,dc=arquitectura,dc=local" -w admin123 | grep "dn:"
dn: dc=arquitectura,dc=local
dn: ou=contratistas,dc=arquitectura,dc=local
dn: ou=directores,dc=arquitectura,dc=local
dn: ou=residentes,dc=arquitectura,dc=local
slave@slave:~/PFASI$

```

Uno de los puntos que se menciona para la implementación del caso práctico, es que el manejo de los planos, maquetas 3D y normativas de construcción tengan un espacio compartido, y donde las versiones estén controladas, la nube privada que escogimos para lograr esto es Nextcloud, y la cuál, aprovechando que en el paso anterior se levantó docker para el uso de OpenLDAP, también lo haremos para la implementación de Nextcloud, así como la instalación de PostgreSQL así como la instalación de PostgreSQL. Para la implementación de la base de datos de Nextcloud, se optó por utilizar PostgreSQL en lugar de las opciones predeterminadas (como SQLite o MariaDB) debido a su alta capacidad para manejar grandes volúmenes de datos y operaciones concurrentes. Al tratarse de un entorno empresarial donde se almacenarán planos arquitectónicos, maquetas 3D y normativas que requieren un estricto control de versiones por parte de múltiples residentes y contratistas, PostgreSQL garantiza un rendimiento óptimo, mayor seguridad y la integridad transaccional necesaria para el trabajo colaborativo en tiempo real.

```

echo "Creando red interna aislada en Docker..."
docker network create red_nextcloud

echo "Desplegando el contenedor de PostgreSQL (Motor de Base de
Datos)..."
docker run -d \
  --name base_datos_postgres \
  --network red_nextcloud \
  --restart always \
  -e POSTGRES_DB=arquitectura \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=root \
  postgres:15

echo "Esperando 5 segundos a que la base de datos esté lista..."
sleep 5

echo "Desplegando el contenedor de Nextcloud (Conectado a
PostgreSQL)..."
docker run -d \
  -p 8080:80 \
  --name nube_arquitectura \
  --network red_nextcloud \
  --restart always \
  nextcloud

echo "Verificando puerto 8080 en el firewall del Slave..."
firewall-cmd --permanent --add-port=8080/tcp 2>/dev/null
firewall-cmd --reload 2>/dev/null

```

Ahora bien, una vez instalado y que ya esté corriendo el servicio de Nextcloud, nos pasamos a la máquina master, ya que hay que darle instrucciones al balanceador de carga (HAProxy), que si alguien entra a la IP 192.168.12.250 por el puerto 8080, mande ese tráfico directo a la nube de Nextcloud en la máquina slave. Para no estar editando el archivo de haproxy.cfg manualmente, se corre todo lo siguiente en la terminal.

```
# 1. Agregamos la configuración de Nextcloud al final del archivo
de HAProxy
sudo bash -c 'cat <<EOF >> /etc/haproxy/haproxy.cfg

frontend nextcloud_front
    bind *:8080
    default_backend nextcloud_back

backend nextcloud_back
    balance roundrobin
    server nodo2_nextcloud 192.168.12.201:8080 check
EOF'

# 2. Abrimos el puerto en el firewall del Master
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload

# 3. Reiniciamos HAProxy para que tome los cambios
sudo systemctl restart haproxy
```

Ya una vez haya quedado la configuración anterior, si vamos a nuestro navegador y buscamos “http://192.168.50.250:8080”, aparecerá lo siguiente:

En esta ventana se definirá al “dueño” de la nube, por ejemplo, “dirgen”, con la contraseña “root” (con fines prácticos). Además, es en este apartado donde se debe enlazar la aplicación de Nextcloud con la base de datos previamente creada en nuestro entorno de Docker. Para ello, en la sección de 'Configuración de la base de datos', se selecciona la opción de PostgreSQL e ingresamos las credenciales definidas en nuestro script: el usuario "admin", la contraseña "root", el nombre de la base de datos "arquitectura" y, como host, el nombre de nuestro contenedor, que

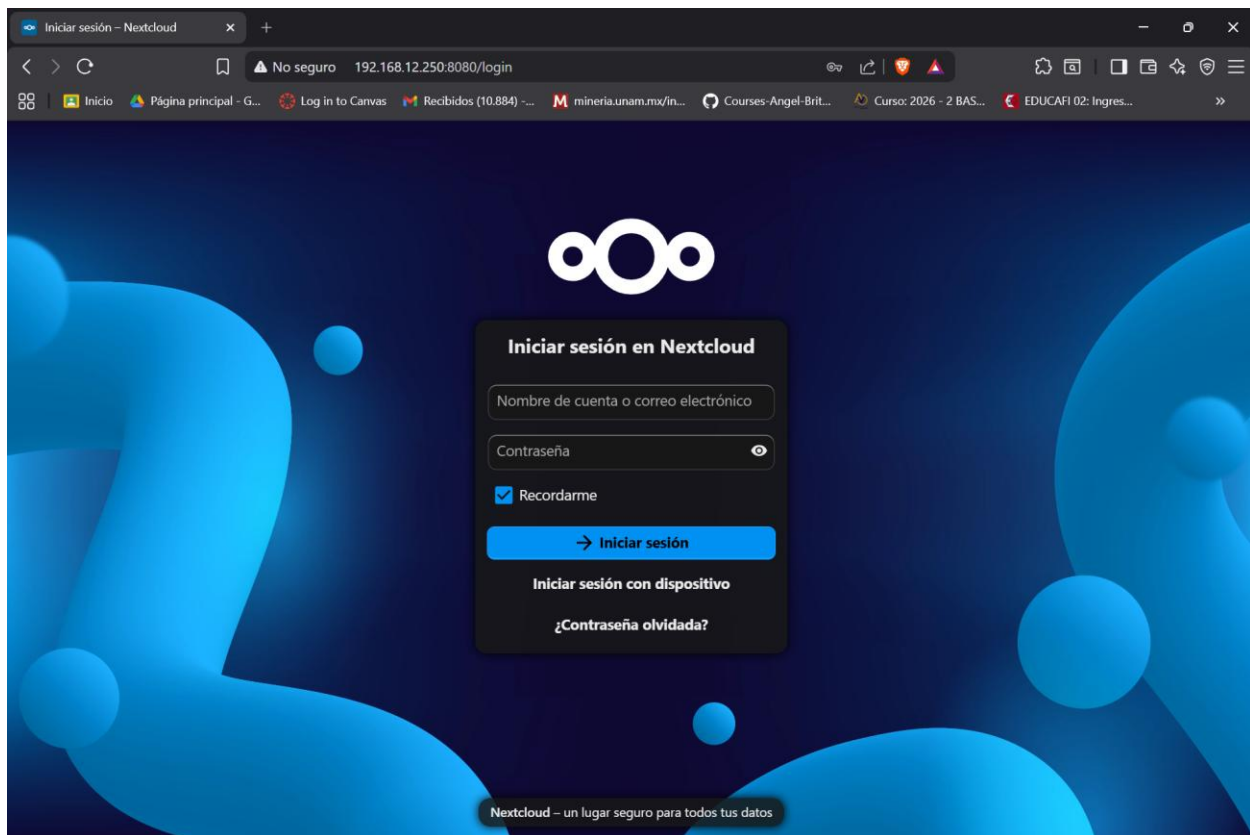
es "bddPostgres". Con esto, aseguramos que la nube guarde toda la información de manera persistente y correctamente vinculada a nuestro motor de base de datos.



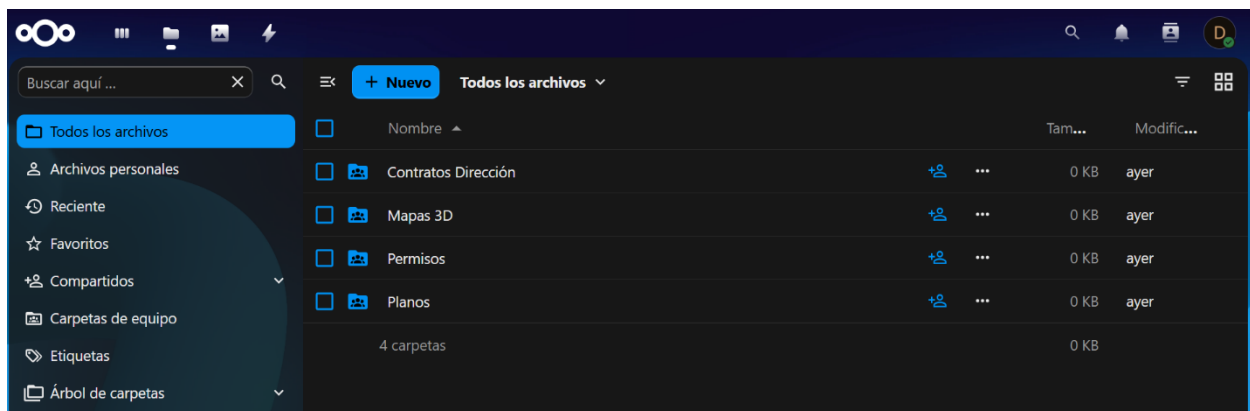
The image shows a web-based configuration form for Nextcloud installation. It is divided into several sections:

- Crear cuenta de administración**: Contains two input fields. The first is labeled "Nombre de la cuenta de administración" with the value "dirgen". The second is labeled "Contraseña de la cuenta de administración" with masked characters "...." and a toggle icon. Below these is a red error message: "La contraseña es muy débil".
- ▼ Almacenamiento y base de datos**: A section header with a dropdown arrow.
- Carpeta de datos**: An input field containing the path "/var/www/html/data".
- Configuración de la base de datos**: A section header.
- Database Selection**: Three buttons labeled "SQLite", "MySQL/MariaDB", and "PostgreSQL". The "PostgreSQL" button is highlighted in blue.
- Usuario de la base de datos**: An input field containing the value "admin".
- Contraseña de la base de datos**: An input field with masked characters "...." and a toggle icon.
- Nombre de la base de datos**: An input field containing the value "arquitectura".
- Host de la base de datos**: An input field containing the value "bddPostgres".

Una vez introducidos los parámetros de configuración y tras finalizar el proceso de instalación, el sistema de Nextcloud procede a inicializar las tablas en la base de datos PostgreSQL y a estructurar el directorio de almacenamiento en el servidor. Al concluir este proceso, la plataforma redirige automáticamente a la pantalla de inicio de sesión. Como se observa a continuación, el portal ya se encuentra completamente operativo y listo para recibir las credenciales de administración previamente definidas, lo que permitirá ingresar al panel principal para comenzar con la estructuración y carga de los documentos, planos y normativas del proyecto.

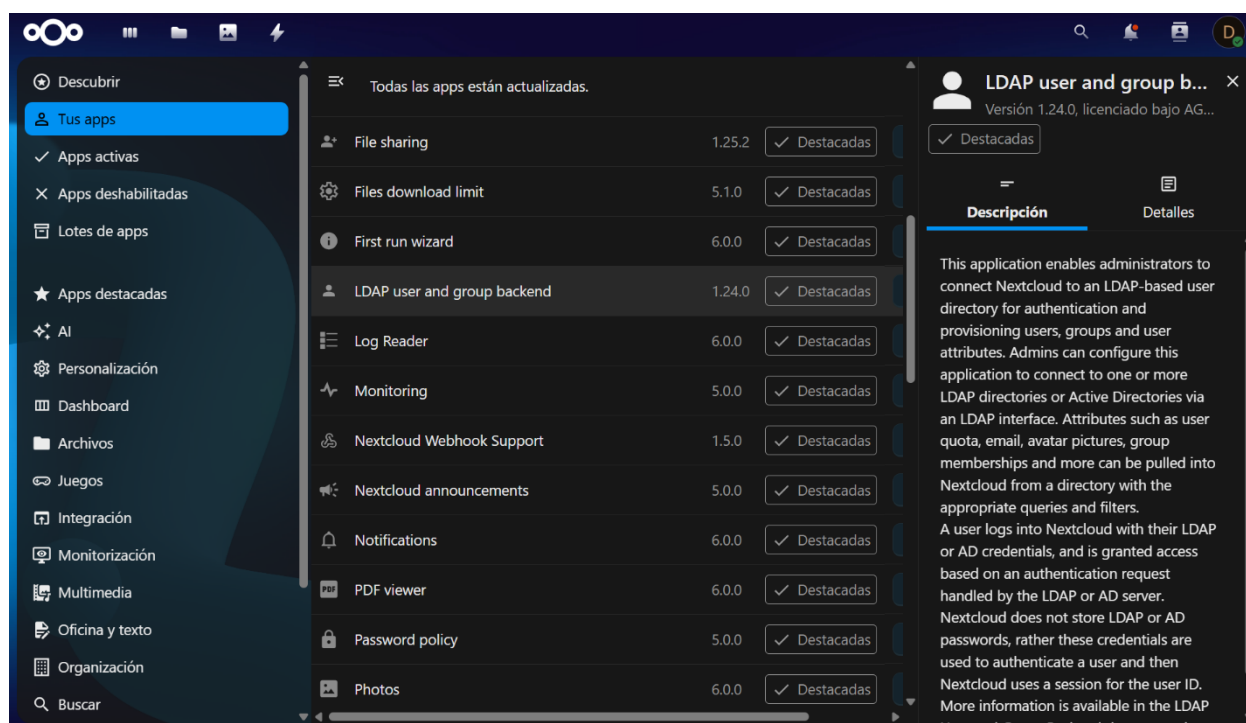


Tras ingresar las credenciales e iniciar sesión con éxito, se accede al panel principal de gestión de archivos de Nextcloud. En cumplimiento con los requerimientos del caso práctico, se procedió a generar la estructura de directorios base para el entorno de la empresa. Como se evidencia en la captura a continuación, se crearon las carpetas dedicadas para albergar las 'Maquetas 3D', las 'Normativas de Construcción' y los 'Planos Estructurales', entre otros. De esta forma, se establece un repositorio centralizado y estructurado, lo cual permitirá un control de versiones eficiente y facilitará el trabajo colaborativo seguro entre la dirección y los contratistas involucrados.

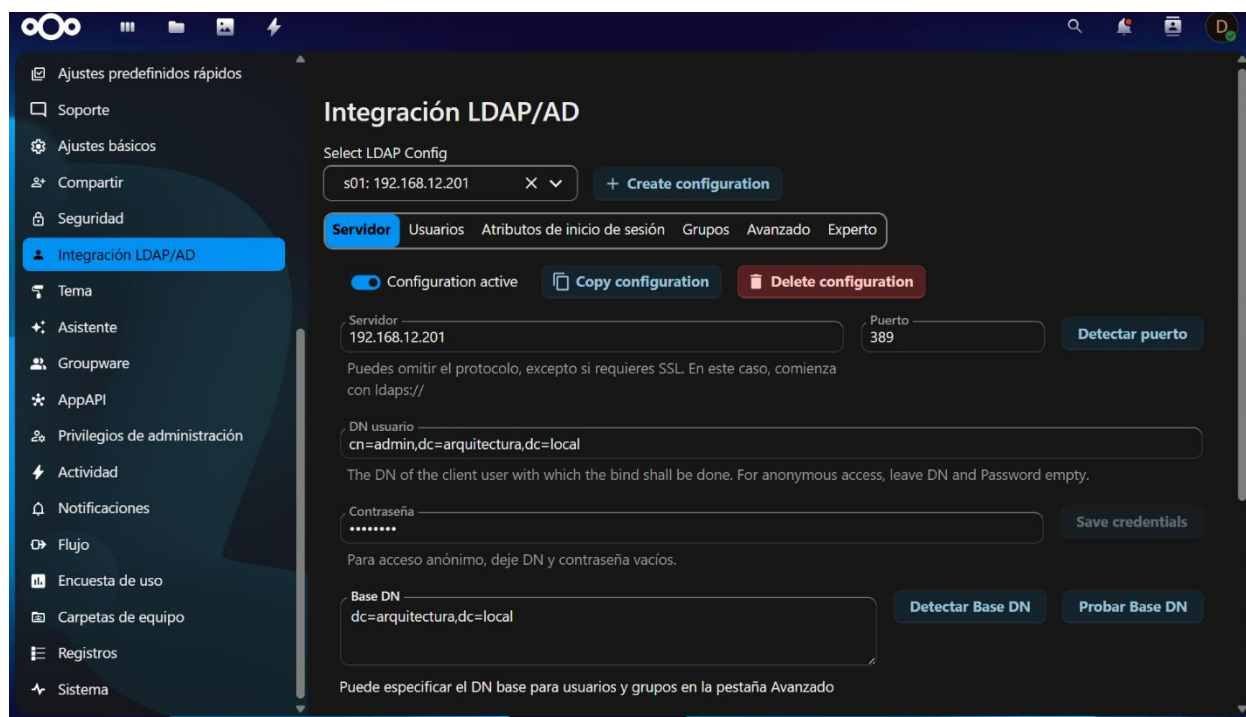


Ya al tener configurado el espacio de Nextcloud, se prosigue con la integración de LDAP en el mismo servicio, para tener un mejor control de los directorios y de los usuarios que vayan a utilizar el sistema. Para esto, Nextcloud cuenta con una herramienta para poder enlazarlos, se

encuentra en el apartado “aplicaciones”, donde, una vez seleccionado aparecerá un menú con diversas herramientas, la que nos interesa para poder integrar LDAP con Nextcloud es “LDAP user and hgroup backend”.



Una vez descargada, y que esté de manera activa, nos dirigimos al apartado de “Configuraciones de administración”, y buscar el apartado de “Integración LDAP/AD”, en la barra de la izquierda.



Aquí, para nuestro caso práctico, implementamos los siguientes archivos .ldif con la configuración correspondiente para poder tener una pequeña base de datos que, de acuerdo a lo solicitado, siendo los siguientes:

- Para la creación de la base (son dos códigos diferentes):

```
#Código 1
dn: dc=arquitectura,dc=local
objectClass: top
objectClass: dcObject
objectClass: organization
o: Firma Arquitectura
dc: arquitectura

dn: cn=Manager,dc=arquitectura,dc=local
objectClass: organizationalRole
cn: Manager
description: Administrador del Directorio

dn: ou=contratistas,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: contratistas

dn: ou=directores,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: directores

dn: ou=residentes,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: residentes
```

```
#Código 2
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=arquitectura,dc=local

replace: olcRootDN
olcRootDN: cn=Manager,dc=arquitectura,dc=local

add: olcRootPW
olcRootPW:

Para la creación de los grupos:
dn: cn=Residentes,ou=residentes,dc=arquitectura,dc=local
objectClass: groupOfNames
cn: Residentes
member: uid=apozos,ou=residentes,dc=arquitectura,dc=local

dn: cn=Directores,ou=directores,dc=arquitectura,dc=local
objectClass: groupOfNames
cn: Directores
member: uid=asilverio,ou=directores,dc=arquitectura,dc=local
```

- Para la alta de los usuarios

```
dn: ou=residentes,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: residentes

dn: ou=directores,dc=arquitectura,dc=local
objectClass: organizationalUnit
ou: directores

dn: uid=apozos,ou=residentes,dc=arquitectura,dc=local
objectClass: inetOrgPerson
uid: apozos
sn: Pozos
cn: Angel Pozos
userPassword: admin123

dn: uid=asilverio,ou=directores,dc=arquitectura,dc=local
objectClass: inetOrgPerson
uid: asilverio
sn: Silverio
cn: Andres Silverio
userPassword: admin123
```

Sin embargo, la creación de estos archivos de manera local, no servirá de nada si no se sube al docker de OpenLDAP, por lo que, en general, se utiliza el siguiente comando:

```
sudo docker cp [NOMBRE DEL ARCHIVO LDIF] servidor_ldap:[NOMBRE DEL ARCHIVO LDIF]
```

El cual copia el archivo creado de manera manual, al Docker “servidor_ldap”.

Y por último, el siguiente comando, el cual hace que el archivo recién creado lo ejecute LDAP, y ya lo pueda reconocer Nextcloud:

```
sudo docker exec servidor_ldap ldapadd -x -D
"cn=admin,dc=arquitectura,dc=local" -w admin123 -f /[NOMBRE DEL ARCHIVO LDIF]
```

Ya una vez ejecutados los comandos, nos regresamos a Nextcloud, donde se pondrá la IP del servidor (en este caso, donde esta instalado el docker con Nextcloud y Olap), y el puerto por defecto (que es 389).

Servidor

192.168.12.201

Puerto

389

Puedes omitir el protocolo, excepto si requieres SSL. En este caso, comienza con ldaps://

De ahí, automáticamente Nextcloud identificará toda nuestra configuración de los archivos creados anteriormente, primero, detectando la base DN, en la misma pantalla de “Servidor”.

Servidor
Usuarios
Atributos de inicio de sesión
Grupos
Avanzado
Experto

☒ Configuration active
Copy configuration
Delete configuration

Servidor
192.168.12.201
Puerto
389
Detectar puerto

Puedes omitir el protocolo, excepto si requieres SSL. En este caso, comienza con ldaps://

DN usuario
cn=admin,dc=arquitectura,dc=local

The DN of the client user with which the bind shall be done. For anonymous access, leave DN and Password empty.

Contraseña
.....
Save credentials

Para acceso anónimo, deje DN y contraseña vacíos.

Base DN
dc=arquitectura,dc=local
Detectar Base DN
Probar Base DN

Después, para verificar que Nextcloud está identificando a los usuarios que se dieron de alta mediante el archivo .ldif, no movemos a la pestaña de “usuarios”, ahí, en el cuadro de “Sólo estas clases de objetos”, se selecciona la opción de “inetOrgPerson”, y después, se le aprieta en el boton de “Verificar configuración y contar usuarios”, apareciendo los que identifica Nextcloud que ya hemos dado de alta.

Servidor
Usuarios
Atributos de inicio de sesión
Grupos
Avanzado
Experto

El listado y la búsqueda de usuarios está restringido a estos criterios:

Sólo estas clases de objetos:
inetOrgPerson X
Los objetos de clases más comunes para los usuarios son organizationalPerson, persona, usuario y inetOrgPerson. Si no está seguro de qué objeto de clase seleccionar, por favor, consulte con su administrador de directorio.

Sólo desde estos grupos:

☐ Editar consulta LDAP
Filtro LDAP: (objectclass=inetOrgPerson)

Verificar configuración y contar usuarios
User count: 6 usuarios encontrados

Configuración de prueba
Ayuda

Luego, pasamos a la pestaña de “Atributos de inicio de sesión”, donde, en el cuadro de selección de “Otros atributos”, se selecciona la opción de “uid”, con el objetivo de que los usuarios inicien sesión con el uid que se configuró en los archivos .ldif, y que, al momento de escribirlo en el cuadro de texto de “Test Login Name”, y verificar que existe, aparece un recuadro validando que si existe.

Select LDAP Config

s01: 192.168.12.201 X v + Create configuration

Usuario encontrado y configuración verificada. X

Servidor Usuarios **Atributos de inicio de sesión** Grupos Avanzado Experto

When logging in, Nextcloud will find the user based on the following attributes:

☒ Nombre de usuario LDAP/AD:
Allows login against the LDAP/AD username, which is either 'uid' or 'sAMAccountName' and will be detected.

☐ Dirección email LDAP/AD:
Allows login against an email attribute. 'mail' and 'mailPrimaryAddress' allowed.

Otros atributos:

uid X v

☐ Editar consulta LDAP

Filtro LDAP: (&(|(objectclass=inetOrgPerson))(|(uid=%uid)(| (uid=%uid))))

Test Login name: asilverio

Verificar configuración

Por último, en la pestaña de “grupos”, en el recuadro de selección de “Sólo estas clases de objetos”, se selecciona “groupOfNames”, para validar cada uno de los grupos que se crearon en los archivos .ldif, y que, al momento de apretar el recuadro de “Verifique los ajustes y cuente los grupos”, aparecerán los grupos encontrados (que son 2):

Servidor Usuarios Atributos de inicio de sesión **Grupos** Avanzado Experto

Groups meeting these criteria are available in Nextcloud:

Sólo estas clases de objetos:

groupOfNames X v

Sólo desde estos grupos:

v

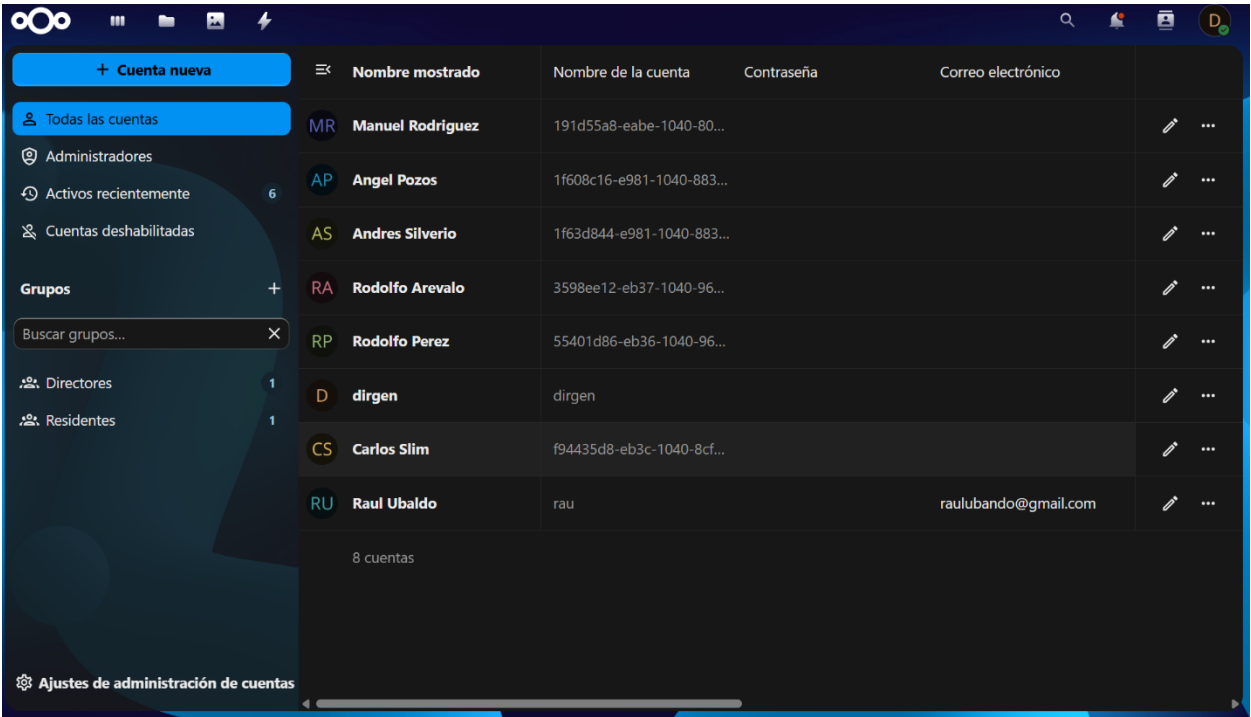
☐ Editar consulta LDAP

Filtro LDAP: (&(|(objectclass=groupOfNames)))

Verifique los ajustes y cuente los grupos 2 grupos encontrados

De igual manera, para validar de una forma más visual que todas estas configuraciones son válidas, si seleccionamos la opción de “cuentas” donde aparecen las opciones del perfil de dirgen, aparecerán todas las cuentas dadas de alta, que son las siguientes (al momento de las

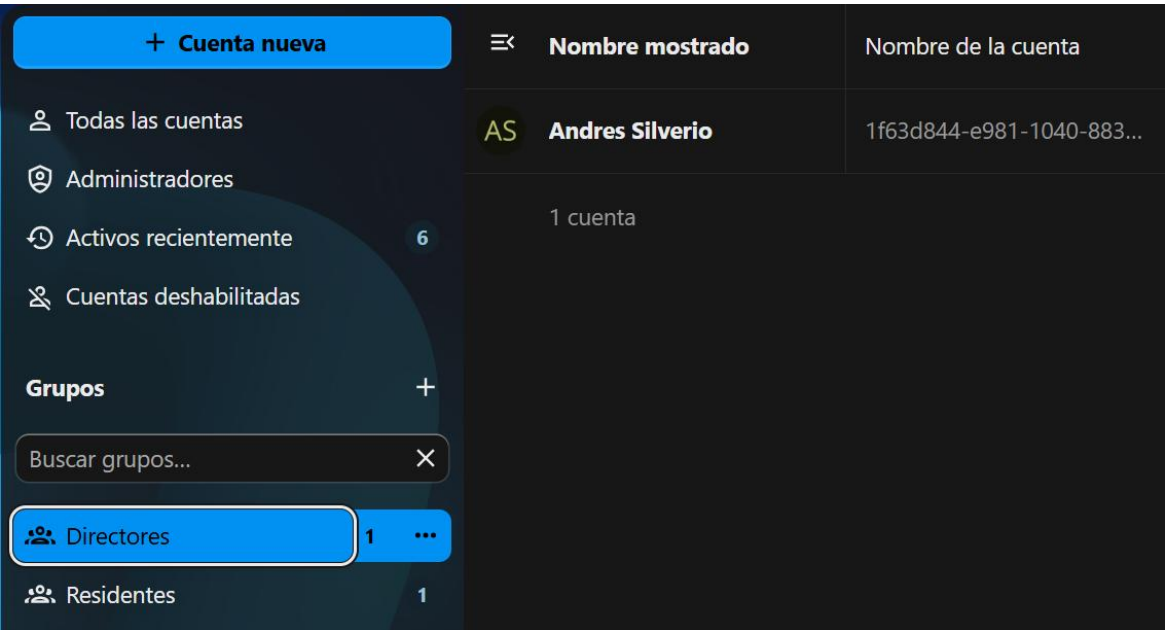
capturas, hay más usuarios porque se hicieron varias pruebas, no porque el sistema no los haya identificado con anterioridad).



The screenshot shows a user management interface with a sidebar on the left and a main table area. The sidebar contains a '+ Cuenta nueva' button, a 'Todas las cuentas' link, and several filter links: 'Administradores', 'Activos recientemente' (with a count of 6), 'Cuentas deshabilitadas', and a 'Grupos' section with a search bar and two items: 'Directores' (1) and 'Residentes' (1). The main table has columns for 'Nombre mostrado', 'Nombre de la cuenta', 'Contraseña', 'Correo electrónico', and an actions column. It lists 8 accounts, including Manuel Rodriguez, Angel Pozos, Andres Silverio, Rodolfo Arevalo, Rodolfo Perez, dirgen, Carlos Slim, and Raul Ubaldo.

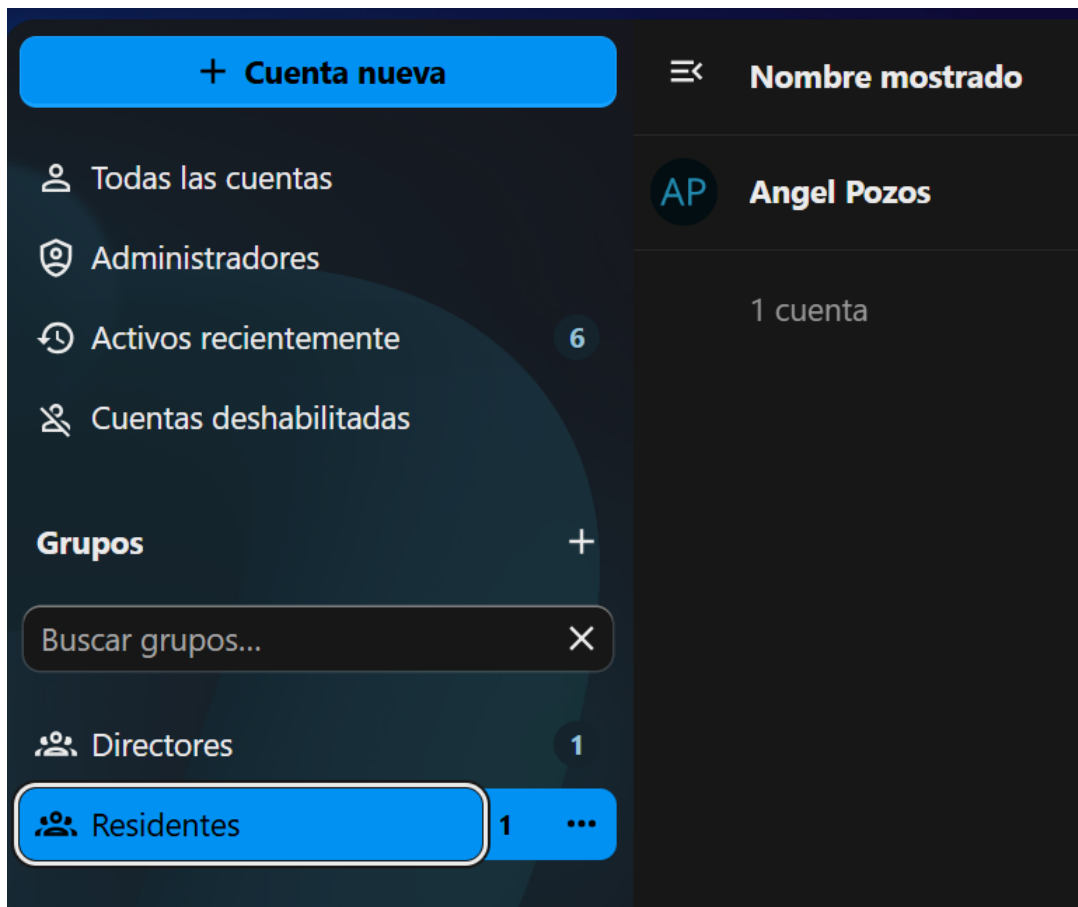
	Nombre mostrado	Nombre de la cuenta	Contraseña	Correo electrónico	
MR	Manuel Rodriguez	191d55a8-eabe-1040-80...			
AP	Angel Pozos	1f608c16-e981-1040-883...			
AS	Andres Silverio	1f63d844-e981-1040-883...			
RA	Rodolfo Arevalo	3598ee12-eb37-1040-96...			
RP	Rodolfo Perez	55401d86-eb36-1040-96...			
D	dirgen	dirgen			
CS	Carlos Slim	f94435d8-eb3c-1040-8cf...			
RU	Raul Ubaldo	rau		raulubando@gmail.com	
8 cuentas					

Y que de igual manera, aparecen ambos grupos también creados mediante los comandos de .ldif, con los miembros respectivos a estos mismos.



This screenshot shows the same interface as the first, but with the 'Directores' group selected in the sidebar. The main table area now displays only one account, 'Andres Silverio', and a message '1 cuenta'.

	Nombre mostrado	Nombre de la cuenta
AS	Andres Silverio	1f63d844-e981-1040-883...
1 cuenta		



Como último servicio a implementar en la máquina virtual 'master', se optó por la configuración de una Red Privada Virtual (VPN) utilizando WireGuard. La finalidad de este servicio es garantizar un acceso remoto cifrado y seguro a la infraestructura interna de la empresa (como la intranet y la nube privada de Nextcloud) para aquellos directores, residentes y contratistas que requieran conectarse desde el exterior. Para agilizar su despliegue y facilitar su gestión, la instalación se realizó mediante contenedores de Docker utilizando la imagen 'wg-easy', la cual provee una interfaz gráfica intuitiva para la administración de los accesos.

```
#!/bin/bash

echo "Instalando Docker en el Master..."
dnf config-manager
--add-repo=https://download.docker.com/linux/centos/docker-ce.repo

dnf install -y docker-ce docker-ce-cli containerd.io --nogpgcheck

echo "Iniciando y habilitando el demonio de Docker..."
systemctl enable --now docker

echo "Desplegando la VPN (wg-easy) con interfaz gráfica..."

docker rm -f vpn_arquitectura 2>/dev/null

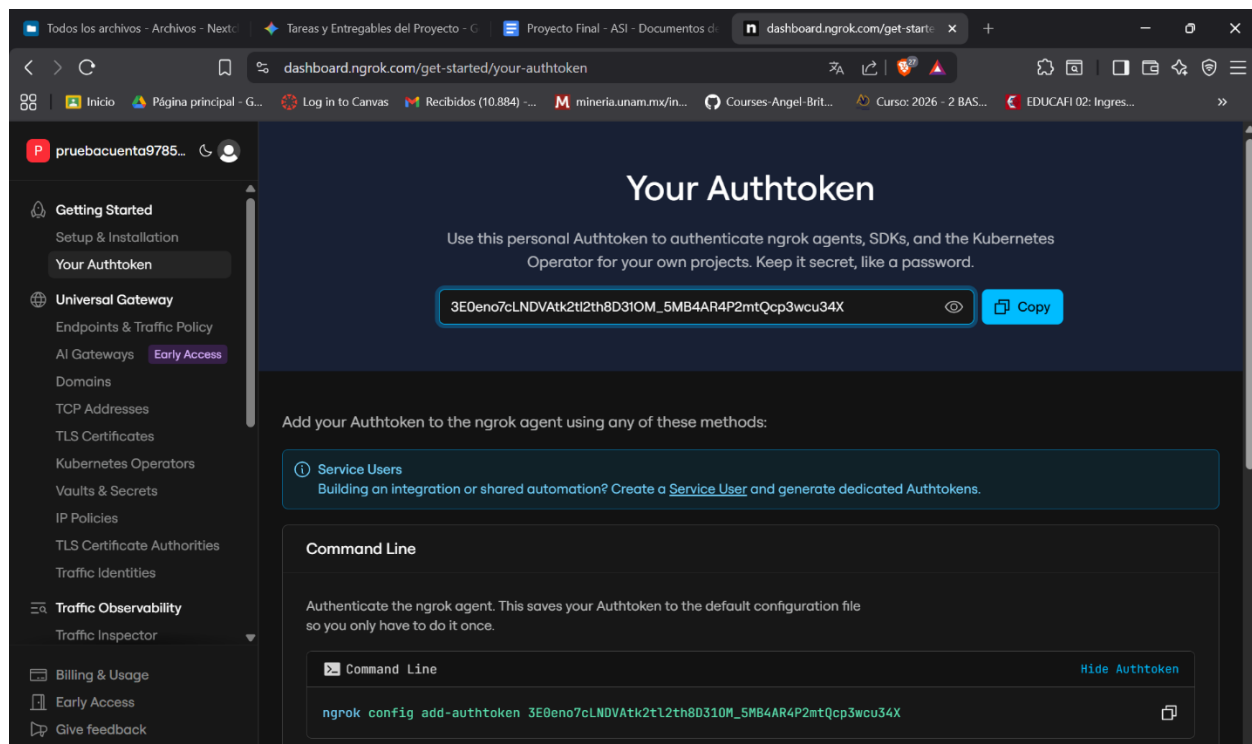
docker run -d \
  --name vpn_arquitectura \
  -e WG_HOST=192.168.12.200 \
  -e PASSWORD=root \
  -v ~/.wg-easy:/etc/wireguard:Z \
  -p 51820:51820/udp \
  -p 51821:51821/tcp \
  --cap-add=NET_ADMIN \
  --cap-add=SYS_MODULE \
  --sysctl="net.ipv4.conf.all.src_valid_mark=1" \
  --sysctl="net.ipv4.ip_forward=1" \
  --restart always \
  weejewel/wg-easy

echo "Abriendo puertos en el firewall (51820 UDP para VPN, 51821 TCP para la Web)..."
firewall-cmd --permanent --add-port=51820/udp
firewall-cmd --permanent --add-port=51821/tcp
firewall-cmd --reload
```

Es importante mencionar una consideración técnica respecto al servicio de la VPN: en caso de que el sistema anfitrión (la máquina física) entre en estado de suspensión o se reinicie, los módulos del kernel necesarios para el ruteo de red interno en Docker pueden llegar a descargarse. Si esto ocurre, el contenedor de WireGuard perderá su conectividad y no podrá enrutar el tráfico correctamente hacia la red virtual. Para solucionarlo y restablecer el servicio de manera rápida, es necesario volver a cargar los módulos de las tablas de IP en el kernel y reiniciar el contenedor ejecutando los siguientes comandos en la terminal de la máquina virtual master:

```
master@master:~/PFASI$ sudo modprobe ip_tables
master@master:~/PFASI$ sudo modprobe iptable_nat
master@master:~/PFASI$ sudo docker restart vpn_arquitectura
vpn_arquitectura
```

Para lograr que los servicios internos sean accesibles desde el exterior sin necesidad de realizar configuraciones complejas de ruteo o apertura de puertos en el firewall perimetral físico, se optó por implementar la herramienta Ngrok. El primer paso consistió en dirigirse a su página web oficial y registrar una cuenta para el proyecto. Una vez completado el registro y validado el acceso, la plataforma proporciona un token de autenticación (Authtoken) único.



A continuación, se descargó e instaló el binario en la máquina virtual "master", y se procedió a vincular nuestra cuenta mediante el token generado utilizando los siguientes comandos:

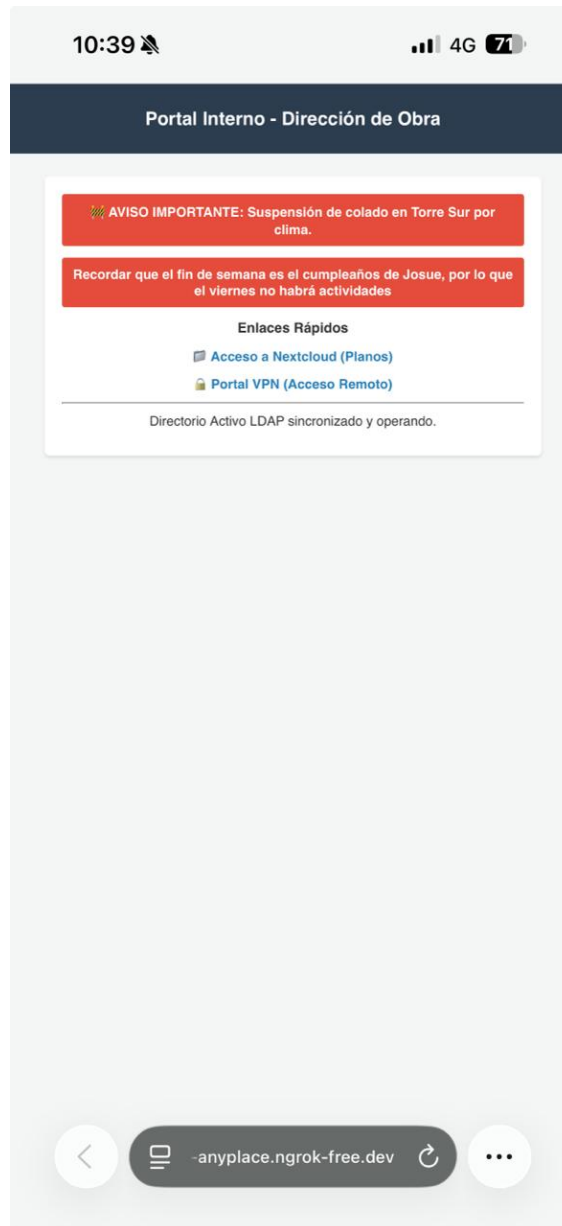
Comandos para la instalación de ngrok:

```
wget
https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-amd64.t
gz
sudo tar xvzf ./ngrok-v3-stable-linux-amd64.tgz -C /usr/local/bin
```

Donde, el comando utilizado para poder acceder a ngrok y crear una URL válida, es el siguiente:

```
master@master:~/PFASI$ ngrok config add-authtoken 3E0eno7cLNDVAtk2t12th8D31OM_5M
B4AR4P2mtQcp3wcu34X
Authtoken saved to configuration file: /home/master/.config/ngrok/ngrok.yml
master@master:~/PFASI$
```

Ya una vez vinculado el token de autenticación de manera exitosa, se procedió a levantar el túnel de Ngrok para exponer nuestro entorno local hacia internet. Al ejecutar el comando de inicio, la plataforma nos proporcionó una URL pública, la cual permite el acceso a nuestra infraestructura (como la intranet o la nube de Nextcloud) desde el exterior, sin necesidad de estar conectados a la red local, tal y como se muestra a continuación.



Si bien el haber configurado el servicio de WireGuard garantiza la creación del túnel cifrado, por el momento este solo permite establecer conexiones si el cliente se encuentra dentro de la misma red local. Para lograr que cualquier usuario autorizado (como directores o contratistas) pueda conectarse a la infraestructura interna y acceder a servicios como Nextcloud de forma remota desde Internet, es necesario ingresar a la configuración del router proveedor (ISP) y crear una regla de redirección de puertos (Port Forwarding o NAT).

Como se ilustra en la configuración de la siguiente imagen, se agregó una regla denominada 'WireGuardVPN'. Esta regla especifica que todo el tráfico externo que reciba el router por el puerto de inicio 51820 a través del protocolo UDP, deberá ser traducido y redirigido a la dirección IP local de la máquina física anfitriona (192.168.1.228) por ese mismo puerto. Con este mapeo, el router permite el paso del tráfico de la VPN, habilitando la conexión remota y segura desde cualquier dispositivo exterior.

Agregar nueva aplicación

Interfaz WAN	Predeterminado
Nombre del Dispositivo	-- Seleccione la dirección IP por los hosts LAN --
Dirección IP del servidor	192 . 168 . 1 . 228
Categoría de aplicación	Regla definida por el usuario
Nombre del Servicio	WireGuardVPN
Puerto de inicio	51820
Puerto final	51820
Puerto de inicio de traducción	51820
Puerto final de traducción	51820
Protocolo	UDP

Cancel OK

NAT

Port Forwarding Activación de puerto DMZ ALG Asignación de direcciones Sesiones

Esta pantalla proporciona un resumen de todas las aplicaciones NAT y su configuración. Puede crear nuevas aplicaciones NAT y el dispositivo Zyxel utilizará la configuración de reenvío de puertos predeterminada para la categoría que seleccionó.

+ Agregar nueva regla

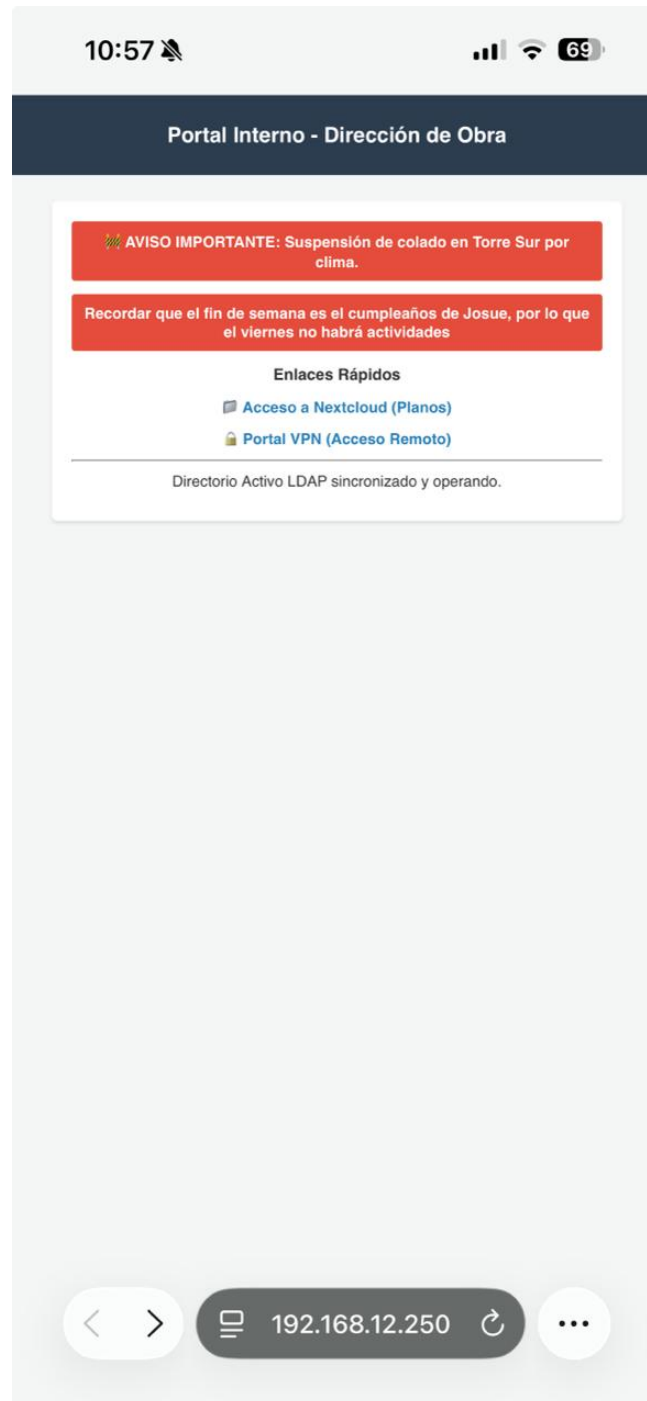
#	Nombre del Servicio	Interfaz WAN	Dirección IP del servidor	Modificar
1	WireGuardVPN	Predeterminado	192.168.1.228	

Finalmente, para comprobar la accesibilidad y el correcto funcionamiento de los servicios desde un dispositivo externo para el trabajo de campo, se configuraron las aplicaciones cliente en un teléfono móvil.

1. Acceso a la VPN (WireGuard): Primero, se descargó la aplicación de WireGuard desde la tienda de aplicaciones del dispositivo. Posteriormente, al ingresar al panel de administración web de wg-easy (a través del puerto 51821 de la IP de nuestro master), se generó un nuevo perfil de cliente, el cual arroja un código QR. Desde la aplicación móvil, se escaneó el código para importar el túnel automáticamente. Al encender la conexión, el teléfono móvil quedó integrado de manera segura a la red local virtual de la empresa.

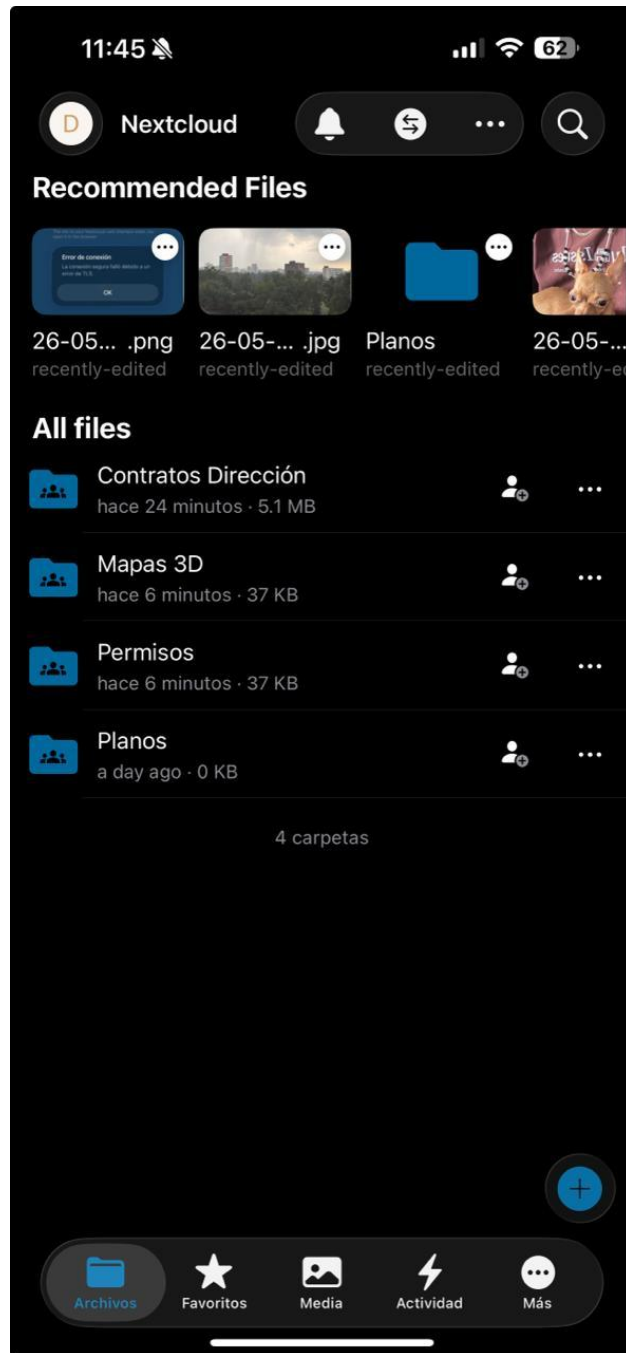


2. Prueba de Internet: Con la VPN activa, se abrió el navegador web del teléfono y se ingresó a la IP virtual (<http://192.168.12.250>). Tras ingresar las credenciales de `htpasswd` correspondientes, se visualizó correctamente el portal de los cronogramas de avance.



3. Acceso a Nextcloud: Se descargó la aplicación móvil oficial de Nextcloud. Al iniciarla, la aplicación solicita la dirección del servidor. Como el dispositivo ya cuenta con acceso a la red local gracias a la VPN, se ingresó la dirección y puerto configurados en el balanceador: <http://192.168.12.250:8080>. Una vez establecida la conexión, se ingresaron

las credenciales de administración (dirgen / root) , otorgando acceso inmediato y sincronización a las carpetas de maquetas 3D, normativas y planos estructurales desde el teléfono celular.



Conclusiones

- Hernández Ruiz Leny Javier: La realización de este proyecto final representó una oportunidad excelente para consolidar de forma práctica y tangible los conceptos de alta disponibilidad, balanceo de carga y seguridad perimetral que revisamos a lo largo del semestre. Desde mi perspectiva, el trabajo enfocado en estructurar la distribución de peticiones mediante el algoritmo round-robin de HAProxy, sumado a la creación del perímetro seguro para el personal en campo a través de la VPN de WireGuard, fue de lo más enriquecedor. Tuvimos que enfrentarnos a retos complejos de redes, como la inestabilidad de las subredes y la configuración del reenvío de puertos (Port Forwarding) dentro de las reglas NAT del módem residencial, pero lograr que un dispositivo móvil externo rompiera el aislamiento y visualizara la intranet local a través del túnel criptográfico UDP validó por completo el diseño de nuestra topología. Considero que logramos entregar una solución de grado empresarial sumamente eficiente que resuelve los requerimientos de la constructora y que demuestra los enormes beneficios operativos y económicos de migrar hacia el uso de software libre y la contenedorización con Docker.
- Silverio Martínez Andrés: Al finalizar este proyecto, me llevo una experiencia muy valiosa sobre lo que verdaderamente implica diseñar, estructurar y estabilizar una red con tolerancia a fallas en el mundo real. En lo personal, el mayor desafío técnico fue lograr que el servidor OpenLDAP se comunicara correctamente con la plataforma de Nextcloud. Lidar con el 'bug' de compatibilidad del atributo memberOf propio de Windows, y tener que intervenir las entrañas del contenedor de la nube mediante la terminal de comandos de Linux (occ) para forzar el mapeo nativo por member, me enseñó que la integración de sistemas heterogéneos requiere un entendimiento profundo del flujo de datos y no solo de la ejecución de scripts automáticos. Asimismo, realizar las pruebas de failover con Keepalived y ver en el monitor continuo cómo el nodo Slave heredaba la IP virtual (192.168.12.250) de manera inmediata al apagar bruscamente el servidor Master, me demostró la efectividad de la alta disponibilidad en nuestra arquitectura.

Bibliografía

- Docker Inc. (s.f.). Docker Documentation. <https://docs.docker.com/>
- Edge Security LLC. (s.f.). WireGuard: Fast, Modern, Secure VPN Tunnel. <https://www.wireguard.com/>
- HAProxy Technologies. (s.f.). HAProxy Documentation. <https://docs.haproxy.org/>
- Keepalived. (s.f.). Keepalived Documentation. <https://keepalived.org/doc/>
- Nextcloud GmbH. (s.f.). Nextcloud Documentation. <https://docs.nextcloud.com/>
- NGINX, Inc. (s.f.). NGINX Documentation. <https://nginx.org/en/docs/>
- Oracle. (s.f.). Oracle VM VirtualBox User Manual. <https://www.virtualbox.org/manual/>
- The CentOS Project. (s.f.). CentOS Documentation. <https://docs.centos.org/>
- The OpenLDAP Project. (s.f.). OpenLDAP Software 2.6 Administrator's Guide. <https://www.openldap.org/doc/admin26/>
- The PostgreSQL Global Development Group. (s.f.). PostgreSQL Documentation. <https://www.postgresql.org/docs/>